


```
fn kruskal(ae:[Edge]) { // SCC, DFS order → reverse gr
  while let Some(u) ← stack.pop() {
    seen[u] = false; n--; comp[u] = 0;
    for source in order.rev() {
      if seen[source] == true {
        comp[u].push(source);
        comp[u].push(comp)
      }
    }
  }
}

Runtime  $\Theta(V + E)$ .
```

11. Union-Find / Disjoint Sets

Forest trees. parent x points upward. Root is representative. rank x upper-bounds height.

```
struct DSU {
  parent: Vec<usize>,
  rank: Vec<usize>,
}

impl DSU {
  fn new(n: DSUParamet: (0..n).collect(),
    rank: Vec<0..n>[] {
    for i in 0..n {
      self.parent[i] = i
      self.rank[i] = 0
    }
  }

  fn find(&self, u: u) {
    if self.parent[u] == u {
      self.parent[u] = self.find(self.parent[u])
    }
    self.parent[u]
  }
}

Complexities:
• linked list weighted union:  $O(m + n \log n)$ 
• forest + union by rank + path compression:  $O(m \alpha(n))$ 
 $\alpha(n) \leq 5$  in practice.
//Cycle detection (undirected):
let mut dsu: DSU = DSU::new(n);
for (u, v) in edges {
  if dsu.union(u, v) {
    // same component
  }
}
```

12. Minimum Spanning Trees

Chooser

sparse graph Kruskal often easiest
dense graph Prim often easier
disconnected graph forests, no MST
MST: spanning tree with min total edge weight
Cut / Cycle Properties

Cur $(S, V - S)$: edge crosses cut iff one endpoint in S , one outside. Light edge = minimum-weight crossing edge.

Cut property: if cut respects chosen edge set A , then a light crossing edge is safe.

Cycle property: for any cycle, a heaviest edge on that cycle is excluded from some MST. If weights distinct, the heaviest edge is in no MST.

Kruskal

Data structures:

```
struct Edge {u,v,w: i32};
let mut edges: Vec<Edge> = ...;
let mut dsu: DSU = DSU::new(n);
let mut mst: Vec<Edge> = Vec::new();
let mut total: i32 = 0;
for e in edges {
  if dsu.union(u, v) {
    // cycle
  }
}
```

Algorithm:

```
fn kruskal(n: edges: Vec<Edge>); // MST
sort_by_weight(edges); dsu=DSU::new(n); total=0; mst=[];
for e in edges {
  if dsu.union(u,v) {
    total+=e.w; mst.push(e);
    if mst.len()==n-1 {break}
  }
}
total,mst
```

Complexity: sort: $O(E \log E)$ DSU: $O(E \alpha(V))$ total: $O(E \log E)$

Output condition: if $mst.len()==n-1$, it is an MST; otherwise graph disconnected and result is a forest.

Traps:

- If graph disconnected $\rightarrow$ MST does not exist; return forest.
- Need sort by weight increasing.
- Skip edge if endpoints already connected.

Prim

```
let mut adj: Vec<Vec<usize, i32>> = vec![Vec::new(); n];
let mut used: Vec<bool> = vec![false; n];
let mut heap: BinaryHeap<Vec<usize, i32>> = BinaryHeap::new();
// (weight, vertex, parent)
fn print(mst: Vec<Edge>); // MST
heap.push((0,0)); total=0; cnt=0;
while let Some((u,v,w))=heap.pop_min() {
  if used[u] {continue}
  used[u]=true; total+=w; cnt+=1;
  for (v,w2) in adj[u] {
    if !used[v] {heap.push((w2,v))}
  }
}
let cnt=cnt-1 // disconnected */
total
```

Complexity:

- Binary heap lazy: $O(E \log E)$; for simple graphs $E \leq V^2$, so $O(E \log V)$.

MST Reduction Stencils

Rounds + shortcuts: add supernode S ; edge S -city cost-airport; old road edges unchanged; MST in augmented graph. Add one new vertex given old MST T ; run MST on T plus new incident edges; old non-tree edges still unnecessary by cut/cycle property. **Points/routes:** complete graph on points, edge cost=squared distance;

```
run Kruskal/Prim.  $E = \Theta(n^2)$ , time  $O(n^2 \log n)$ .
```

13. Shortest Paths

Chooser

unweighted BFS
DAG topu DP
weights ≥ 0 Dijkstra
negative edges Bellman-Ford

Relaxation

Relax edge (u,v,w) : if $dist[u]=\infty$ and $dist[v] > dist[u]+w$, set $dist[v]=dist[u]+w$ and $parent[v]=u$.

Dijkstra

```
fn dijkstra(ae:[Edge], source:usize) {
  // all weights >= 0
  n=ae.len(); dist=vec![INF; n];
  parent=vec![None; n];
  heap.push((0,source));
  while let Some((d,u))=heap.pop_min() {
    if dist[u]<=d {continue}
    dist[u]=d;
    for (v,w) in ae[u] {
      if dist[v]>dist[u]+w {
        dist[v]=dist[u]+w;
        parent[v]=u;
        heap.push((dist[v],v));
      }
    }
  }
}
```

Relax edge (u,v,w) : if $dist[u]=\infty$ and $dist[v] > dist[u]+w$, set $dist[v]=dist[u]+w$ and $parent[v]=u$.

Bellman-Ford

Online 'edges: EdgeList'. Negative edges OK; detects reachable negative cycles.

```
fn bellman_ford(n: edges: source: {
  dist=vec![INF; n]; parent=vec![None; n];
  dist[source]=0;
  for u in 0..n-1 {
    for (v,w) in edges {
      if dist[v]>dist[u]+w {
        dist[v]=dist[u]+w;
        parent[v]=u;
        changed=true;
      }
    }
  }
  if changed {break}
}
```

Runtime $\Theta(V \cdot E)$. Returns 'None' iff a reachable negative cycle is detected.

Negative cycle not reachable from source: Bellman-Ford still returns shortest paths. Only improves matters after pass V reachable from source matter.

Weight transform trap: $w'(e) = a \cdot w(e)$ with $a > 0$ preserves shortest paths.

```
w'(e) = w(e) + a does not preserve shortest paths when candidate paths use different numbers of edges.
```

Shortest Paths in DAG

```
fn dag_shortest(ae:[Edge], source:usize) {
  for (v,w) in adj[u] { // topological order.
    if dist[v]>dist[u]+w { // one relaxation per edge
      dist[v]=dist[u]+w; // negative weights ok
      parent[v]=u;
    }
  }
}
```

Relax edge (u,v,w) : if $dist[u]=\infty$ and $dist[v] > dist[u]+w$, set $dist[v]=dist[u]+w$ and $parent[v]=u$.

Bellman-Ford

Online 'edges: EdgeList'. Negative edges OK; detects reachable negative cycles.

```
fn bellman_ford(n: edges: source: {
  dist=vec![INF; n]; parent=vec![None; n];
  dist[source]=0;
  for u in 0..n-1 {
    for (v,w) in edges {
      if dist[v]>dist[u]+w {
        dist[v]=dist[u]+w;
        parent[v]=u;
        changed=true;
      }
    }
  }
  if changed {break}
}
```

Runtime $\Theta(V \cdot E)$.

Time-Dependent / Ferry Shortest Path

$dist[u] =$ earliest arrival. Road relax: $dist[v] = \min(dist[v], dist[u] + w)$. Ferry/event (u, u, dep, arr) usable iff $dist[u] \leq dep$, then relax $dist[v] = \min(dist[v], arr)$. Use PQ by earliest arrival if time is FIFO/nonnegative; otherwise expand state/event graph.

14. Flow Networks

Directed $G = (V, E)$, source s , sink t , capacities $c(u, v) \geq 0$. Flow constraints: $0 \leq f(u,v) \leq c(u,v)$; conservation for $u \neq s,t$: inflow-outflow; $f = \text{out}(s) = \text{in}(t)$. Augmenting path: $s \rightarrow t$ path in residual graph; bottleneck = $\min c_f(e)$ on path. Flow reduction recipe: selected object = unit flow; capacities = usage limits; source/sink = constraints; value = objective; integral caps $\Rightarrow$ integral solution.

Ford-Fulkerson / Edmonds-Karp

Types: 'cap[u][v]' residual capacity matrix; 'adj: Graph' stores forward+reverse residual neighbors.

```
fn add_edge(u:usize, v:usize, c:i32) {
  adj[u].push((v,c)); adj[v].push((u,-c)); cap[u][v]=c;
}

fn bfs_path(source:usize, sink:usize) {
  let mut queue: Vec<usize> = Vec::new();
  queue.push(source); queue.push(sink);
  while let Some(u)=queue.pop_front() {
    for v in adj[u] {
      if parent[v].is_none() && cap[u][v]>0 {
        parent[v]=u; queue.push(v);
      }
    }
  }
}
```

Build network: add $S \rightarrow L$, original edges $L \rightarrow R, R \rightarrow t$, all capacities 1. Integral max-flow gives integral solution. Capacity 1 forces each left/right vertex used at most once. Unit-flow edge $i \rightarrow T$ is a matched pair; every matching gives same-size flow. Thus max flow = max matching.

Q9 (E12). What does **Longest Path** return on an all-equal array, and how to fix it?

With condition $\leq pivot$, every element is moved left, so $q = r$. Split sizes $n-1$ and 0, hence $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$. Fix: 3-way partition into $<, =, >$ pivot, or split equal keys evenly / return middle of equal block.

Q10 (L19 exam). Ski resort: lifts/slopes have signed altitude changes. Verify every closed tour returns to start w/out net change.

Build graph: stations = vertices, lift/slope = directed edge weighted by altitude change. Bad tour $\leftrightarrow$ directed cycle with nonzero total weight. Run Bellman-Ford from super-source on weights W and $-W$: a positive cycle in W is negative in $-W$; a negative cycle in W is detected in W . Relaxation on pass V in either run $\rightarrow$ nonzero cycle exists. Runtime $O(V \cdot E)$.

Q11 (L19, 10) Min crossing edge of any cut in the MST. (b) Min edge of any cycle is not in MST. Assume distinct weights. (a) Cut property: if lightest crossing edge e absent, add e to MST; cycle contains another crossing edge f with $w(f) > w(e)$; swap $f \rightarrow e$ gives cheaper tree, contradiction. (b) If heaviest cycle edge e in MST, remove it; the cycle has lighter edge f crossing the induced cut; swap $e \rightarrow f$ gives cheaper tree, contradiction.

Q12. Travel passes DE

Travel days $d_1 < \dots < d_n$, pass lengths $L \in \{1, 7, 30\}$, costs c_L .

State $dp[i][L] =$ min cost to cover $d_1 \dots d_n$. Base $dp[n+1] = 0$. Let $next(i, L) =$ first j with $d_j \geq d_i + L$. Recurrence $dp[i] = \min_{L \in \{1,7,30\}} (c_L + dp[next(i, L)])$. Order $i = n..1$. Base $dp[1]$. Runtime $O(n \log n)$ with lower bound. Trap: pass covers days $< d_i + L$, not $\leq d_i + L$.

Q13. Weighted interval scheduling.

Sort intervals by finish time. $p(i) =$ last interval finishing $\leq start_i$. State $dp[i] =$ best weight using first i intervals. Base $dp[0] = 0$. Recurrence $dp[i] = \max(dp[i-1], w_i + dp[p(i)])$. Ans: $dp[n]$. Runtime $O(n \log n)$ with binary search for $p(i)$. Correctness: optimum either excludes interval i or includes it, then previous chosen interval must be $\leq p(i)$.

Q14. Interval scheduling greedy.

Sort by increasing finish time. Take interval if $start \leq last_finish$. Proof: let e be earliest finishing interval. For any optimum O , replace the first interval of O by e ; finish time does not increase, so all later intervals remain feasible. Therefore some optimum contains e . Induct on remaining intervals.

Q15. DP design answer skeleton.

State $dp[s] =$ optimum value for subproblem s . Base: smallest/empty cases. Recurrence $dp[s] = \text{opt}_{c \in valid(s)} F(s, c, dp[next(s, c)])$. Order: dependencies first. Ans: explicitly write returned state. Runtime = #states $\times$ transition cost. Correctness: induction in fill order; recurrence enumerates all valid choices; dependencies optimal by IH.

Q16a. Parametric search / split array into k contiguous parts minimizing max load.

Bounds: $L = \max_{i=1..k} a_i$, $R = \sum_{i=1..k} a_i$. feasible (M) : scan left-to-right, start new part when current sum $+a_i > M$. Count parts. If count $\leq k$, M feasible. Binary search smallest feasible M . Correctness: greedy assigns minimum possible number of parts for fixed M ; feasibility monotone. Runtime $O(n \log \sum a_i)$.

Q16b. Forbidden mutual reachability pairs in directed graph.

Compute SCCs in $O(V + E)$. For each forbidden pair (u, v) , reject iff $comp[u] = comp[v]$. If same SCC, then $u \rightarrow v$ and $v \rightarrow u$. If mutually reachable, they must be in same SCC. Thus all constraints checked exactly.

Q16c. Capacitated activity assignment.

Network: $S \rightarrow$ each student cap 1; student $\rightarrow$ acceptable activity cap 1; activity $\rightarrow$ t cap number of spots. Run maxflow. Ans: feasible iff flow value = #students. Correctness: integral unit leaving each student chooses one acceptable activity; activity capacity enforces spots; any assignment gives same flow.

Q16d. Binary tree diameter / longest path.

Postorder. Empty height = -1 . For node u : $h(u) = 1 + \max(h_L, h_R)$ and $diam(u) = \max(diam_L, diam_R, h_L + h_R + 2)$. Ans: $diam(root)$. Correctness: every longest path either stays inside one child subtree or crosses through U using deepest node on each side.

Q16e. Connected graph with 71 vertices and 71 edges: count spanning trees.

Such graph has exactly one cycle. Find the cycle by DFS back edge or by repeatedly deleting degree-1 leaves. If cycle length is 71 , answer is C . Reason: a spanning tree has $71-1$ edges, so delete one edge. Deleting a cycle edge keeps connected and removes the only cycle; deleting a non-cycle edge disconnects.

Q16f. Coin-row 2-player minimax DP

$dp[i][j][z] =$ max value current player can guarantee from coins $i..j$. Base $dp[i][i][i] = a_i$. Recurrence $dp[i][j][j] = \max(a_i + \min(dp[i+2][j], dp[i+1][j-1]), a_j + \min(dp[i+1][j-1], dp[i][j-2]))$. Opponent chooses the branch minimizing our future gain. Fill by interval length.

Q17. Competitive ratio template.

Define $OPT(I, t)$. Compute $ALG(I)$. Split into exhaustive cases. In each case prove $ALG(I) \leq r \cdot OPT(I, t) + c$. Take worst case. Conclusion: algorithm is r -competitive. Lower bound: adversary chooses input after seeing deterministic algorithm's action.

Q18. Secretary problem.

Random order. Reject first $\lfloor n/e \rfloor$ candidates, remember best seen, then hire first later candidate better than all previous. Success probability tends to $1/e$. Reason: best candidate must appear after sample, and best candidate before it must lie inside sample.

Q19. Online bipartite matching.

Left side known, right vertices arrive online. Greedy: match arriving vertex to any free neighbor. Each greedy edge can be charged by at most two OPT edges, so $ALG \geq \frac{1}{2}$ OPT. Ranking: random rank/permutation on left vertices; match arriving vertex to lowest-ranked free neighbor. Expected $(1 - 1/e)$ OPT.

Q20. Prophet inequality threshold algorithm.

Known independent values X_i . Let $M = \max_i X_i$, $z = E[M]$, threshold

For $n \geq 2|a|$: $\frac{1}{2}n \leq n + a \leq 2n$, hence $(\frac{1}{2})^n b \leq (n+a)^b \leq 2^n b$. Both sides are constants times n^b ; therefore $(n+a)^b = \Theta(n^b)$.

Q3 (E22). Solve $T(n) = 8T(n/2) + \Theta(n^2)$ by recursion tree.

Level i : 8^i nodes, each cost $c(n/2^i)^2$, total $c n^2 2^i$. Sum $\sum_{i=0}^{\log_2 n} c n^2 2^i = \Theta(n^2 \cdot n) = \Theta(n^3)$. Leaves: $n \cdot 8 = n^3$. Hence $T(n) = \Theta(n^3)$.

Q4 (E22). Recursive insertion sort recurrence.

$T(n) = T(n-1) + \Theta(n)$. Unroll: $T(n) = c \sum_{i=1}^n i = c n(n+1)/2 = \Theta(n^2)$.

Q5 (E66). n restaurants on a line, positions M_i , profit P_i , any two at distance $\geq k$. Max profit.

Sort by position. State $dp[i] =$ best profit using restaurants $1..n$. Let $next(i) =$ first $j > i$ with $M_j \geq M_i + k$. Base $dp[n+1] = 0$. Recurrence $dp[i] = \max(dp[i+1], P_i + dp[next(i)])$. Order $i = n..1$.

Ans: $dp[1]$. Runtime $O(n \log n)$ with binary search, $O(n^2)$ if scanning. Correctness: optimum either skips i or takes i ; after taking i , remaining choices start at $next(i)$.

Q5b (E66). Restaurant leftmost-open variant.

Sort positions. $c[i] =$ best total profit if i is leftmost opened. Then $c[i] = P_i + \max\{c[j] : j > i \wedge M_j \geq M_i + k\}$. Ans: $\max_i c[i]$. Bottom-up $O(n^2)$, with binary search/suffix maxima $O(n \log n)$. Fixing the leftmost open restaurant leaves an independent feasible suffix.

Q6 (E65). $O(\log T)$ test whether sorted $A[1..n]$ contains k .

Binary search. Invariant: if K occurs, then $k \in A[\lfloor r \rfloor]$. Compare k with $A[mid]$ and keep only the half that can contain k . Sortedness preserves invariant. Termination: range empty $\rightarrow$ absent, or equality found. Runtime $T(n) = T(n/2) + O(1) = O(\log n)$.

Q7 (E69). Maximum bipartite matching as a flow.

Build network: add $S \rightarrow L$, original edges $L \rightarrow R, R \rightarrow t$, all capacities 1. Integral max-flow gives integral solution. Capacity 1 forces each left/right vertex used at most once. Unit-flow edge $i \rightarrow T$ is a matched pair; every matching gives same-size flow. Thus max flow = max matching.

Q9 (E12). What does **Longest Path** return on an all-equal array, and how to fix it?

With condition $\leq pivot$, every element is moved left, so $q = r$. Split sizes $n-1$ and 0, hence $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$. Fix: 3-way partition into $<, =, >$ pivot, or split equal keys evenly / return middle of equal block.

Q10 (L19 exam). Ski resort: lifts/slopes have signed altitude changes. Verify every closed tour returns to start w/out net change.

Build graph: stations = vertices, lift/slope = directed edge weighted by altitude change. Bad tour $\leftrightarrow$ directed cycle with nonzero total weight. Run Bellman-Ford from super-source on weights W and $-W$: a positive cycle in W is negative in $-W$; a negative cycle in W is detected in W . Relaxation on pass V in either run $\rightarrow$ nonzero cycle exists. Runtime $O(V \cdot E)$.

Q11 (L19, 10) Min crossing edge of any cut in the MST. (b) Min edge of any cycle is not in MST. Assume distinct weights. (a) Cut property: if lightest crossing edge e absent, add e to MST; cycle contains another crossing edge f with $w(f) > w(e)$; swap $f \rightarrow e$ gives cheaper tree, contradiction. (b) If heaviest cycle edge e in MST, remove it; the cycle has lighter edge f crossing the induced cut; swap $e \rightarrow f$ gives cheaper tree, contradiction.

Q12. Travel passes DE

Travel days $d_1 < \dots < d_n$, pass lengths $L \in \{1, 7, 30\}$, costs c_L .

State $dp[i][L] =$ min cost to cover $d_1 \dots d_n$. Base $dp[n+1] = 0$. Let $next(i, L) =$ first j with $d_j \geq d_i + L$. Recurrence $dp[i] = \min_{L \in \{1,7,30\}} (c_L + dp[next(i, L)])$. Order $i = n..1$. Base $dp[1]$. Runtime $O(n \log n)$ with lower bound. Trap: pass covers days $< d_i + L$, not $\leq d_i + L$.

Q13. Weighted interval scheduling.

Sort intervals by finish time. $p(i) =$ last interval finishing $\leq start_i$. State $dp[i] =$ best weight using first i intervals. Base $dp[0] = 0$. Recurrence $dp[i] = \max(dp[i-1], w_i + dp[p(i)])$. Ans: $dp[n]$. Runtime $O(n \log n)$ with binary search for $p(i)$. Correctness: optimum either excludes interval i or includes it, then previous chosen interval must be $\leq p(i)$.

Q14. Interval scheduling greedy.

Sort by increasing finish time. Take interval if $start \leq last_finish$. Proof: let e be earliest finishing interval. For any optimum O , replace the first interval of O by e ; finish time does not increase, so all later intervals remain feasible. Therefore some optimum contains e . Induct on remaining intervals.

Q15. DP design answer skeleton.

State $dp[s] =$ optimum value for subproblem s . Base: smallest/empty cases. Recurrence $dp[s] = \text{opt}_{c \in valid(s)} F(s, c, dp[next(s, c)])$. Order: dependencies first. Ans: explicitly write returned state. Runtime = #states $\times$ transition cost. Correctness: induction in fill order; recurrence enumerates all valid choices; dependencies optimal by IH.

Q16a. Parametric search / split array into k contiguous parts minimizing max load.

Bounds: $L = \max_{i=1..k} a_i$, $R = \sum_{i=1..k} a_i$. feasible (M) : scan left-to-right, start new part when current sum $+a_i > M$. Count parts. If count $\leq k$, M feasible. Binary search smallest feasible M . Correctness: greedy assigns minimum possible number of parts for fixed M ; feasibility monotone. Runtime $O(n \log \sum a_i)$.

Q16b. Forbidden mutual reachability pairs in directed graph.

Compute SCCs in $O(V + E)$. For each forbidden pair (u, v) , reject iff $comp[u] = comp[v]$. If same SCC, then $u \rightarrow v$ and $v \rightarrow u$. If mutually reachable, they must be in same SCC. Thus all constraints checked exactly.

Q16c. Capacitated activity assignment.

Network: $S \rightarrow$ each student cap 1; student $\rightarrow$ acceptable activity cap 1; activity $\rightarrow$ t cap number of spots. Run maxflow. Ans: feasible iff flow value = #students. Correctness: integral unit leaving each student chooses one acceptable activity; activity capacity enforces spots; any assignment gives same flow.

Q16d. Binary tree diameter / longest path.

Postorder. Empty height = -1 . For node u : $h(u) = 1 + \max(h_L, h_R)$ and $diam(u) = \max(diam_L, diam_R, h_L + h_R + 2)$. Ans: $diam(root)$. Correctness: every longest path either stays inside one child subtree or crosses through U using deepest node on each side.

Q16e. Connected graph with 71 vertices and 71 edges: count spanning trees.

Such graph has exactly one cycle. Find the cycle by DFS back edge or by repeatedly deleting degree-1 leaves. If cycle length is 71 , answer is C . Reason: a spanning tree has $71-1$ edges, so delete one edge. Deleting a cycle edge keeps connected and removes the only cycle; deleting a non-cycle edge disconnects.

Q16f. Coin-row 2-player minimax DP

$dp[i][j][z] =$ max value current player can guarantee from coins $i..j$. Base $dp[i][i][i] = a_i$. Recurrence $dp[i][j][j] = \max(a_i + \min(dp[i+2][j], dp[i+1][j-1]), a_j + \min(dp[i+1][j-1], dp[i][j-2]))$. Opponent chooses the branch minimizing our future gain. Fill by interval length.

Q17. Competitive ratio template.

Define $OPT(I, t)$. Compute $ALG(I)$. Split into exhaustive cases. In each case prove $ALG(I) \leq r \cdot OPT(I, t) + c$. Take worst case. Conclusion: algorithm is r -competitive. Lower bound: adversary chooses input after seeing deterministic algorithm's action.

Q18. Secretary problem.

Random order. Reject first $\lfloor n/e \rfloor$ candidates, remember best seen, then hire first later candidate better than all previous. Success probability tends to $1/e$. Reason: best candidate must appear after sample, and best candidate before it must lie inside sample.

Q19. Online bipartite matching.

Left side known, right vertices arrive online. Greedy: match arriving vertex to any free neighbor. Each greedy edge can be charged by at most two OPT edges, so $ALG \geq \frac{1}{2}$ OPT. Ranking: random rank/permutation on left vertices; match arriving vertex to lowest-ranked free neighbor. Expected $(1 - 1/e)$ OPT.

Q20. Prophet inequality threshold algorithm.

Known independent values X_i . Let $M = \max_i X_i$, $z = E[M]$, threshold

$\tau = z/2$. Accept first value at least τ . Guarantee $E[ALG] \geq z/2$. Write proof with $q = P[\text{no value reaches } \tau]$ and overhots $(X_i - \tau)^+$; threshold contribution plus overshoot contribution gives half of prophet value.

Q21. Bounder + postorder tree reconstruction.

root=last(postorder). Split in-order at root into left/right. Sizes determine postorder left/right blocks. Recurse. Correctness by induction on subtree size. Runtime $O(n)$ with key $\rightarrow$ in-order-index map.

Q22. Longest path passing through root.

Compute subtree heights. Answer $h(left) + h(right) + 2$ with empty height -1 . Correct because path through root must be deepest node in left side + root + deepest node in right side.

Q23. Domino/grid matching.

Checkerboard color cells. Put black cells in R , white cells in R , edge iff adjacent. A domino is one matched edge; no cell reused because matching. Max domino placement = max bipartite matching.